UNIVERSIDAD PRIVADA DEL VALLE

Base de Datos III (BDD3T4) · Avance 2 · Semana 1

DOCUMENTO DE DISEÑO DEL PAQUETE PKG_TURISMOVIVE

Grupo 6 — TurismoVive · Septiembre de 2026

1. Propósito del paquete

El paquete PKG_TURISMOVIVE agrupa la lógica de negocio operativa de TurismoVive sobre la Capa Transaccional construida en el Avance 1 (tipos de objeto, Nested Table, tablas de objetos y REF). Encapsula en un solo componente las reglas que hoy están dispersas en consultas sueltas: cálculo de tarifas y comisiones, disponibilidad de cupos en tiempo real, registro y ciclo de vida de las reservas, y consulta de paquetes flexibles. Cada subprograma responde a un eje de la Pregunta Rectora:

Eje de la Pregunta Rectora	Subprogramas que lo cubren
Calcular tarifas dinámicas y comisiones	`fn_precio_paquete`, `fn_total_reserva`
Mostrar disponibilidad de cupos en tiempo real	`fn_cupos_disponibles`, `sp_registrar_reserva`, `sp_cambiar_estado_reserva`
Armar paquetes turísticos flexibles	`sp_listar_actividades`, `sp_resumen_paquete`
Anticipar la demanda estacional	Fuera del alcance de esta semana (capa analítica de avances posteriores). Las reservas registradas por el paquete, con fecha y destino, dejan los datos base para ese análisis.

2. Esquema reutilizado (Avance 1, sin modificaciones)

El script del paquete no ejecuta ningún CREATE/ALTER sobre tipos ni tablas. Se apoya en los siguientes objetos del Avance 1:

- `tipo_paquete_turistico` / `tabla_paquetes_turisticos`: se reutiliza el método `calcula_precio_total()` (no se duplica la fórmula) y la Nested Table `actividades` (`actividades_nt` de `tipo_actividad`).
- `tipo_viajero` / `tabla_viajeros`: origen de la referencia `ref_viajero`.
- `tipo_reserva` / `tabla_reservas`: tabla transaccional con REF y SCOPE FOR; el paquete inserta y actualiza aquí.

Decisiones derivadas de no modificar el esquema:

- El esquema no almacena la capacidad de cada paquete. Se define la constante pública `c_cupo_maximo = 20`, parametrizable en `fn_cupos_disponibles`; en un avance posterior puede migrar a un atributo del tipo sin cambiar la firma.
- El informe de la Semana 3 menciona `TABLA_HOTELES_ASOCIADOS`, pero no existe en el script del Avance 1; por ello los hoteles quedan fuera del alcance del paquete.

3. Especificación de subprogramas

Subprograma	Tipo	Parámetros de entrada	Salida	Descripción y reglas
`fn_precio_paquete`	Función	`p_id_paquete` NUMBER	NUMBER — precio final del paquete	Recupera el objeto con `VALUE()` y devuelve `calcula_precio_total()`. Lanza `e_paquete_inexistente`.
`fn_total_reserva`	Función	`p_id_reserva` NUMBER	NUMBER — importe total de la reserva	Navega con `DEREF(ref_paquete)` y multiplica el precio por `num_pasajeros`. Lanza `e_reserva_inexistente`; detecta REF dangling.
`fn_cupos_disponibles`	Función	`p_id_paquete` NUMBER; `p_cupo_maximo` NUMBER = `c_cupo_maximo`	NUMBER — cupos libres (nunca negativo)	cupo máximo − Σ `num_pasajeros` de las reservas PENDIENTE/CONFIRMADA del paquete (filtro con DEREF en el WHERE). Valida que el paquete exista.

Subprograma	Tipo	Parámetros de entrada	Salida	Descripción y reglas
sp_registrar_reserva	Procedimiento	p_id_reserva, p_id_viajero, p_id_paquete, p_num_pasajeros NUMBER; p_fecha_reserva DATE = TRUNC(SYSDATE); p_estado VARCHAR2 = 'PENDIENTE'	— Inserta en tabla_reservas y confirma por DBMS_OUTPUT	Valida pasajeros > 0, estado permitido, existencia de viajero y paquete, fecha anterior a fecha_inicio y cupo suficiente; obtiene REF(v) y REF(p) e inserta con el constructor tipo_reserva. Excepciones: e_dato_invalido, e_viajero_inexistente, e_paquete_inexistente, e_sin_cupo, e_reserva_duplicada.
sp_cambiar_estado_reserva	Procedimiento	p_id_reserva NUMBER; p_nuevo_estado VARCHAR2	— Actualiza estado_reserva	Normaliza a mayúsculas, valida que el estado sea PENDIENTE, CONFIRMADA o CANCELADA y actualiza; si SQL%ROWCOUNT = 0 lanza e_reserva_inexistente. Cancelar libera cupo automáticamente.
sp_listar_actividades	Procedimiento	p_id_paquete NUMBER	— Listado por DBMS_OUTPUT	Aplana la Nested Table con TABLE(p.actividades) y muestra cada actividad, la suma de costos, el precio base, la comisión y el precio total.
sp_resumen_paquete	Procedimiento	p_id_paquete NUMBER	OUT: p_destino VARCHAR2, p_num_actividades NUMBER, p_precio_total NUMBER, p_cupos_disponibles NUMBER	Devuelve un resumen por parámetros OUT reutilizando las funciones del paquete; protege contra COLLECTION_IS_NULL si la colección es atómicamente nula.

Subprogramas privados (solo en el cuerpo): obtener_paquete(p_id_paquete) RETURN tipo_paquete_turistico, que centraliza el SELECT VALUE(p) INTO y convierte NO_DATA_FOUND en e_paquete_inexistente para todo el paquete; y estado_valido(p_estado) RETURN BOOLEAN.

4. Manejo de excepciones

Las seis excepciones propias se declaran en la especificación y se asocian a un código con PRAGMA EXCEPTION_INIT. Así el cuerpo las lanza con RAISE_APPLICATION_ERROR (mensaje descriptivo en SQLERRM) y el cliente puede capturarlas por nombre, por ejemplo WHEN pkg_turismovive.e_sin_cupo THEN. Los códigos −20010 a −20015 no chocan con el −20001 que usa el constructor del Avance 1.

Excepción	Código	Origen	Se lanza cuando
e_paquete_inexistente	ORA-20010	Propia, desde NO_DATA_FOUND	El id de paquete no existe, o la REF de la reserva quedó dangling.
e_viajero_inexistente	ORA-20011	Propia, desde NO_DATA_FOUND	El id de viajero no existe.
e_reserva_inexistente	ORA-20012	Propia, desde NO_DATA_FOUND o SQL%ROWCOUNT = 0	No existe la reserva consultada o actualizada.
e_reserva_duplicada	ORA-20013	Propia, desde DUP_VAL_ON_INDEX	Se intenta insertar un id_reserva que ya existe (clave primaria).

Excepción	Código	Origen	Se lanza cuando
e_sin_cupo	ORA-20014	Propia (regla de negocio)	Los pasajeros solicitados superan los cupos disponibles del paquete.
e_dato_invalido	ORA-20015	Propia (validación)	Pasajeros ≤ 0, fecha nula o posterior al inicio del viaje, o estado no permitido.

Excepciones predefinidas manejadas: NO_DATA_FOUND (en obtener_paquete, fn_total_reserva y en la búsqueda del viajero) y DUP_VAL_ON_INDEX (en sp_registrar_reserva). Ambas se capturan y se relanzan como excepciones propias con un mensaje de negocio, en lugar de dejar que el cliente reciba un ORA-01403 u ORA-00001 genérico.

5. Decisiones de diseño

- **Sin COMMIT ni ROLLBACK dentro del paquete.** El control de la transacción queda en el cliente, lo que permite agrupar varias llamadas y usar SAVEPOINT en las pruebas.
- **Validar antes del DML.** Todas las reglas se verifican antes del INSERT/UPDATE: si una falla, no se toca ninguna fila.
- **Un solo punto de verdad para tarifas.** El paquete invoca calcula_precio_total() del tipo en lugar de repetir la fórmula; si cambia la política de comisiones, cambia en un solo lugar.
- **Solo las reservas activas consumen cupo.** PENDIENTE y CONFIRMADA descuentan; CANCELADA no. Cancelar una reserva libera cupo sin lógica adicional.
- **Parámetros por defecto** (fecha, estado, cupo máximo) para simplificar las llamadas habituales sin perder flexibilidad.

6. Pruebas ejecutadas

Todas están incluidas en Avance2_Semana1_PKG_TURISMOVIVE.sql y se ejecutan dentro de un SAVEPOINT que se revierte al final, de modo que los datos del Avance 1 quedan intactos.

#	Prueba	Resultado esperado
4.1	fn_precio_paquete(1..3) comparado con p.calcula_precio_total()	1573 / 868 / 1728, idénticos al método del objeto
4.2	fn_cupos_disponibles(1..3) con los datos del Avance 1	18 / 19 / 16
4.3	sp_listar_actividades(2) y sp_resumen_paquete(1)	3 actividades listadas; Salar de Uyuni, 2 actividades, 1573, 18 cupos
4.4	Registrar reserva 4 (viajero 3, paquete 2, 3 pasajeros) → confirmar → calcular total → cancelar	Cupos 19 → 16; total 868 × 3 = 2604; cupos vuelven a 19
5.1–5.8	Ocho bloques que provocan cada excepción (paquete/viajero/reserva inexistente, duplicada, sin cupo, fecha, pasajeros y estado inválidos) y la capturan por nombre	Mensaje “Capturada …” con el texto de SQLERRM en cada caso
6–7	ROLLBACK TO SAVEPOINT; consulta a USER_OBJECTS y USER_PROCEDURES	3 reservas; PACKAGE y PACKAGE BODY en estado VALID; 7 subprogramas públicos